Firmas digitales

Suponga que Alicia ordena un automóvil rojo a Bob. Bob quiere conservar la orden para que un tercero pueda comprobar su origen. Alicia firmará con su llave privada y la verificación se hará con su llave pública.

En este cuaderno firmaremos la orden con RSA y ECDSA. Ejecute las celdas en orden y explique los resultados en los espacios previstos. No necesita archivos ni resultados de otra práctica.

```
import hashlib
import hmac
import secrets
import importlib.util
import subprocess
import sys

if importlib.util.find_spec("cryptography") is None:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "cryptography"])

from cryptography.exceptions import InvalidSignature
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import ec, padding, rsa, utils

def hash_entero(mensaje):
    return int.from_bytes(hashlib.sha256(mensaje).digest(), "big")

print("Entorno preparado.")
```

```
Entorno preparado.
```

1. Firmar y verificar una orden con RSA

Diapositivas 2–8. Un esquema de firma tiene algoritmos de generación de llaves, firma y verificación. Si las llaves corresponden y el mensaje no cambia, la firma generada debe verificar.

Empezaremos con números pequeños para observar la operación RSA: $\sigma = y^d \bmod n$ y $y = \sigma^e \bmod n$. Este cálculo ilustra el álgebra; no es un esquema seguro para firmar mensajes.

```
p, q = 61, 53
n, e = p*q, 17
d = pow(e, -1, (p-1)*(q-1))
orden = b"orden de un automovil rojo"
y = hash_entero(orden) % n
sigma = pow(y, d, n)
recuperado = pow(sigma, e, n)
print("Representante:", y, "| firma:", sigma, "| verificación:", recuperado)
assert (y, sigma, recuperado) == (1323, 2577, 1323)
```

```
Representante: 1323 | firma: 2577 | verificación: 1323
```

Firma con la biblioteca

RSA-PKCS1-v1.5 codifica el hash y su identificador antes de aplicar RSA. La biblioteca prepara esa codificación al firmar y la comprueba al verificar.

Ahora conservaremos la firma y cambiaremos el color del automóvil en la orden.

```
privada_rsa = rsa.generate_private_key(public_exponent=65537, key_size=2048)
publica_rsa = privada_rsa.public_key()
firma_rsa = privada_rsa.sign(orden, padding.PKCS1v15(), hashes.SHA256())

def verificar_rsa(mensaje, firma):
    try:
        publica_rsa.verify(firma, mensaje, padding.PKCS1v15(), hashes.SHA256())
        return True
    except InvalidSignature:
```

```
        return False

print("Orden original:", verificar_rsa(orden, firma_rsa))
print("Orden modificada:", verificar_rsa(b"orden de un automovil azul", firma_rsa))
assert verificar_rsa(orden, firma_rsa)
assert not verificar_rsa(b"orden de un automovil azul", firma_rsa)
```

```
Orden original: True
Orden modificada: False
```

1. Relacione `y`, `sigma` y `recuperado` con las operaciones RSA de las diapositivas. ¿Qué operación usa la llave privada y cuál la pública?
2. Compruebe la misma firma con otra orden usando `verificar_rsa`. ¿Por qué no basta conservar la firma si cambia el mensaje?
3. Si dos mensajes distintos tuvieran el mismo hash, ¿qué ocurriría con una firma que se calcula a partir de ese resumen?

Respuesta.

Escriba aquí su explicación.

2. Firmar y verificar con ECDSA

Diapositivas 9–12. Sea G un punto de orden primo q. La llave privada es α y la pública es $U = \alpha G$. Para firmar se usa un valor secreto por mensaje α_t, a partir del cual se obtiene el par (r, s). Ese valor no debe reutilizarse para mensajes distintos.

Usaremos P-256 con SHA-256. La biblioteca realiza las operaciones sobre la curva y devuelve el par en una codificación que podemos abrir con `decode_dss_signature`.

Verificaremos la firma con el mensaje original, con un mensaje modificado, con otra llave pública y con una componente s igual a cero.

```
curva = ec.SECP256R1()
privada_ec = ec.generate_private_key(curva)
publica_ec = privada_ec.public_key()
firma_ec = privada_ec.sign(orden, ec.ECDSA(hashes.SHA256()))
r, s = utils.decode_dss_signature(firma_ec)

def verificar_ec(publica, mensaje, firma):
    try:
        publica.verify(firma, mensaje, ec.ECDSA(hashes.SHA256()))
        return True
    except InvalidSignature:
        return False

otra_publica = ec.generate_private_key(curva).public_key()
firma_s_cero = utils.encode_dss_signature(r, 0)
print("Orden original:", verificar_ec(publica_ec, orden, firma_ec))
print("Orden modificada:", verificar_ec(publica_ec, orden + b"!", firma_ec))
print("Otra llave pública:", verificar_ec(otra_publica, orden, firma_ec))
print("Componente s igual a cero:", verificar_ec(publica_ec, orden, firma_s_cero))
assert utils.encode_dss_signature(r, s) == firma_ec
assert verificar_ec(publica_ec, orden, firma_ec)
assert not verificar_ec(publica_ec, orden + b"!", firma_ec)
assert not verificar_ec(otra_publica, orden, firma_ec)
assert not verificar_ec(publica_ec, orden, firma_s_cero)
```

```
Orden original: True
Orden modificada: False
Otra llave pública: False
Componente s igual a cero: False
```

1. ¿Qué información necesita el verificador? ¿Cómo sabe que la llave pública que está usando corresponde a Alicia?
2. En la verificación se calcula $a = h(m)s^{-1} \bmod q$ y $b = rs^{-1} \bmod q$. Use $U = \alpha G$ y la ecuación de s para explicar por qué $aG + bU = \alpha_t G$ en una firma correcta.
3. ¿Por qué no se puede realizar esa verificación con $s = 0$?

Respuesta.

Escriba aquí su explicación.

Referencias

- RFC 8017: RSA-PKCS1-v1.5.
- NIST FIPS 186-5: ECDSA.
- Biblioteca cryptography: firmas RSA y ECDSA.